Acticity 1 - Socratic Prompting:

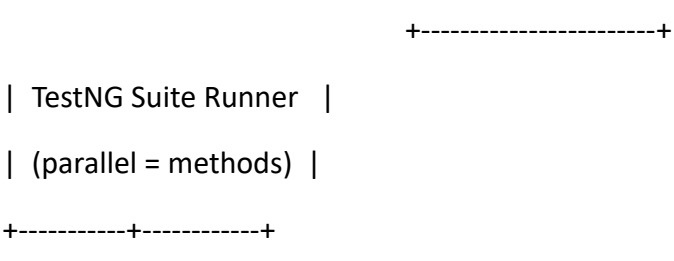
Answer:

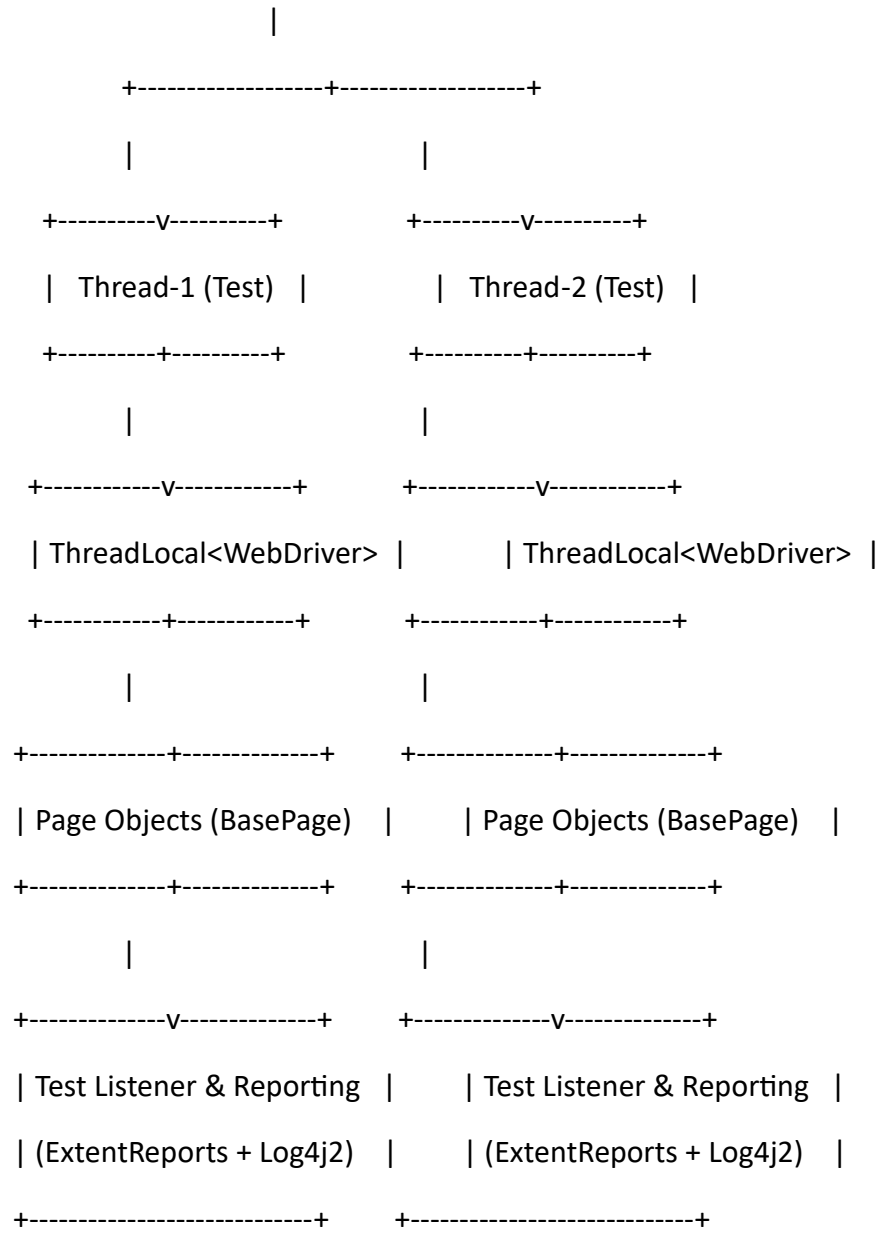
Below is the complete design blueprint and production-ready implementation for your scalable, data-driven Selenium framework.

1. Summary of Requirements

Requirement Area	Selected Choice	Strategy / Implementation Details
Data Management	Apache POI (.xlsx)	Test datasets driven by Excel spreadsheets using an Apache POI utility integrated with TestNG @DataProvider.
Driver & Execution	Local Parallel Execution	ThreadLocal<WebDriver> thread isolation, TestNG parallel execution (parallel="methods"), and Selenium 4 Manager for automatic driver bin management.
Reporting & Logging	ExtentReports + Log4j2	ExtentReports V5 for execution metrics and failure screenshots, paired with Log4j2 for console and rolling file logs.
Config Management	Multi-Environment .properties	Environment-aware property loading driven by -Denv (e.g., qa, dev) defaulting to qa.

2. Architecture & Design Decisions





* Thread-Safety & Isolation (ThreadLocal): Prevents cross-thread interference during parallel execution by giving each test thread its own isolated instance of WebDriver and ExtentTest.

* Page Object Model with Base Abstraction: All page classes extend a BasePage that encapsulates explicit waits (WebDriverWait), reducing boilerplate and flakiness.

* Decoupled Data Engine: Test logic remains clean while an ExcelUtil converts Excel tables into 2D arrays (Object[][]) consumed directly by TestNG @DataProvider.

* Listener-Driven Lifecycle Management: ExtentReports and Log4j2 hooks are managed through TestNG's ITestListener, keeping test classes free of reporting boilerplate.

3. Recommended Project Structure

```
selenium-automation-framework/
├── pom.xml
├── src/
|   ├── main/
|   |   └── java/
|   |       └── com/
|   |           └── framework/
|   |               ├── config/
|   |               |   └── ConfigReader.java
|   |               ├── driver/
|   |               |   ├── DriverFactory.java
|   |               |   └── DriverManager.java
|   |               ├── utils/
|   |               |   ├── ExcelUtil.java
|   |               |   ├── ScreenshotUtil.java
|   |               |   └── Log.java
|   |               ├── reports/
|   |               |   ├── ExtentManager.java
|   |               |   └── ExtentTestManager.java
|   |               └── pages/
|   |                   ├── BasePage.java
|   |                   └── LoginPage.java
```

```
|   └── test/
|       ├── java/
|       |   └── com/
|       |       └── framework/
|       |           ├── listeners/
|       |           |   └── TestListener.java
|       |           └── tests/
|       |               ├── BaseTest.java
|       |               └── LoginTest.java
|       └── resources/
|           ├── config/
|           |   ├── dev.properties
|           |   └── qa.properties
|           ├── testdata/
|           |   └── TestData.xlsx
|           ├── log4j2.xml
|           └── testng.xml
```

4. Complete Implementation

4.1. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                            http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.framework</groupId>
<artifactId>selenium-automation-framework</artifactId>
<version>1.0-SNAPSHOT</version>

<properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <selenium.version>4.23.0</selenium.version>
    <testng.version>7.10.2</testng.version>
    <poi.version>5.3.0</poi.version>
    <extent.version>5.1.2</extent.version>
    <log4j.version>2.23.1</log4j.version>
</properties>

<dependencies>
    <!-- Selenium Java -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>${selenium.version}</version>
    </dependency>

    <!-- TestNG -->
    <dependency>
```

```
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>${testng.version}</version>
    <scope>test</scope>
</dependency>

<!-- Apache POI for Excel Parsing -->
<dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>${poi.version}</version>
</dependency>

<!-- ExtentReports -->
<dependency>
    <groupId>com.aventstack</groupId>
    <artifactId>extent-reports</artifactId>
    <version>${extent.version}</version>
</dependency>

<!-- Log4j2 -->
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-api</artifactId>
    <version>${log4j.version}</version>
</dependency>
```

```
    <dependency>
      <groupId>org.apache.logging.log4j</groupId>
      <artifactId>log4j-core</artifactId>
      <version>${log4j.version}</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.3.1</version>
        <configuration>
          <suiteXmlFiles>
            <suiteXmlFile>src/test/resources/testng.xml</suiteXmlFile>
          </suiteXmlFiles>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

4.2. Configuration Management

src/test/resources/config/qa.properties

```
browser=chrome
```

```
baseUrl=https://opensource-demo.orangehrmlive.com
implicitWait=10
explicitWait=15
headless=false
```

src/test/resources/config/dev.properties

```
browser=firefox
baseUrl=https://dev-demo.orangehrmlive.com
implicitWait=5
explicitWait=10
headless=false
```

src/main/java/com/framework/config/ConfigReader.java

```
package com.framework.config;

import java.io.FileInputStream;
import java.io.IOException;
import java.util.Properties;

/**
 * Singleton Configuration Reader dynamically loading target environment property files.
 */
public class ConfigReader {
    private static Properties properties;

    static {
```

```
    // Default to 'qa' environment if -Denv is not supplied
    String env = System.getProperty("env", "qa").toLowerCase();
    String configPath = "src/test/resources/config/" + env + ".properties";

    try (FileInputStream fis = new FileInputStream(configPath)) {
        properties = new Properties();
        properties.load(fis);
    } catch (IOException e) {
        throw new RuntimeException("Could not load configuration file at: " + configPath, e);
    }
}

public static String getProperty(String key) {
    String value = properties.getProperty(key);
    if (value == null) {
        throw new RuntimeException("Property key '" + key + "' not found in configuration.");
    }
    return value;
}

public static int getIntProperty(String key) {
    return Integer.parseInt(getProperty(key));
}

public static boolean getBooleanProperty(String key) {
    return Boolean.parseBoolean(getProperty(key));
```

```
    }
}
```

4.3. Logging Configuration

src/test/resources/log4j2.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
    <Appenders>
        <Console name="ConsoleAppender" target="SYSTEM_OUT">
            <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss.SSS} [%t] %-5level %logger{36} -
%msg%n"/>
        </Console>
        <RollingFile name="FileAppender" fileName="logs/automation.log"
                     filePattern="logs/automation-%d{yyyy-MM-dd}-%i.log">
            <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss.SSS} [%t] %-5level %logger{36} -
%msg%n"/>
            <Policies>
                <SizeBasedTriggeringPolicy size="10MB"/>
            </Policies>
        </RollingFile>
    </Appenders>
    <Loggers>
        <Root level="info">
            <AppenderRef ref="ConsoleAppender"/>
            <AppenderRef ref="FileAppender"/>
        </Root>
    </Loggers>
```

```
</Configuration>
```

src/main/java/com/framework/utils/Log.java

```
package com.framework.utils;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

public class Log {
    private static final Logger logger = LogManager.getLogger(Log.class);

    public static void info(String message) {
        logger.info(message);
    }

    public static void warn(String message) {
        logger.warn(message);
    }

    public static void error(String message) {
        logger.error(message);
    }

    public static void debug(String message) {
        logger.debug(message);
    }
```

```
}
```

4.4. Driver Management (ThreadLocal)

src/main/java/com/framework/driver/DriverManager.java

```
package com.framework.driver;

import org.openqa.selenium.WebDriver;

/**
 * Thread-safe wrapper around ThreadLocal<WebDriver>.
 */
public class DriverManager {
    private static final ThreadLocal<WebDriver> driverThreadLocal = new ThreadLocal<>();

    public static WebDriver getDriver() {
        return driverThreadLocal.get();
    }

    public static void setDriver(WebDriver driver) {
        driverThreadLocal.set(driver);
    }

    public static void unload() {
        driverThreadLocal.remove();
    }
}
```

src/main/java/com/framework/driver/DriverFactory.java

```
package com.framework.driver;

import com.framework.config.ConfigReader;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.edge.EdgeOptions;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.firefox.FirefoxOptions;

import java.time.Duration;

public class DriverFactory {

    public static WebDriver createDriver() {
        WebDriver driver;
        String browser = ConfigReader.getProperty("browser").toLowerCase();
        boolean isHeadless = ConfigReader.getBooleanProperty("headless");

        switch (browser) {
            case "chrome":
                ChromeOptions chromeOptions = new ChromeOptions();
                if (isHeadless) chromeOptions.addArguments("--headless=new");
```

```
            chromeOptions.addArguments("--start-maximized");
            driver = new ChromeDriver(chromeOptions);
            break;

        case "firefox":
            FirefoxOptions firefoxOptions = new FirefoxOptions();
            if (isHeadless) firefoxOptions.addArguments("-headless");
            driver = new FirefoxDriver(firefoxOptions);
            driver.manage().window().maximize();
            break;

        case "edge":
            EdgeOptions edgeOptions = new EdgeOptions();
            if (isHeadless) edgeOptions.addArguments("--headless=new");
            driver = new EdgeDriver(edgeOptions);
            driver.manage().window().maximize();
            break;

        default:
            throw new IllegalArgumentException("Unsupported browser: " + browser);
    }

    int implicitWait = ConfigReader.getIntProperty("implicitWait");
    driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(implicitWait));

    return driver;
```

```
    }
}
```

4.5. Reporting & Utilities

src/main/java/com/framework/reports/ExtentManager.java

```
package com.framework.reports;

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;
import com.aventstack.extentreports.reporter.configuration.Theme;

public class ExtentManager {
    private static ExtentReports extent;

    public static synchronized ExtentReports getInstance() {
        if (extent == null) {
            ExtentSparkReporter spark = new
ExtentSparkReporter("build/reports/ExtentReport.html");
            spark.config().setReportName("Automation Test Results");
            spark.config().setDocumentTitle("Execution Report");
            spark.config().setTheme(Theme.STANDARD);

            extent = new ExtentReports();
            extent.attachReporter(spark);
            extent.setSystemInfo("Framework", "Selenium Java Data-Driven");
            extent.setSystemInfo("OS", System.getProperty("os.name"));
```

```
        }
        return extent;
    }
}
```

src/main/java/com/framework/reports/ExtentTestManager.java

```
package com.framework.reports;

import com.aventstack.extentreports.ExtentTest;

/**
 * Thread-safe ExtentTest container.
 */
public class ExtentTestManager {
    private static final ThreadLocal<ExtentTest> extentTestMap = new ThreadLocal<>();

    public static ExtentTest getTest() {
        return extentTestMap.get();
    }

    public static void setTest(ExtentTest test) {
        extentTestMap.set(test);
    }

    public static void unload() {
        extentTestMap.remove();
```

```
    }
}
```

src/main/java/com/framework/utils/ScreenshotUtil.java

```
package com.framework.utils;

import com.framework.driver.DriverManager;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;

public class ScreenshotUtil {

    /**
     * Captures screenshot as Base64 for clean ExtentReport inline embedding.
     */
    public static String getBase64Screenshot() {
        return ((TakesScreenshot)
DriverManager.getDriver()).getScreenshotAs(OutputType.BASE64);
    }
}
```

src/main/java/com/framework/utils/ExcelUtil.java

```
package com.framework.utils;

import org.apache.poi.ss.usermodel.*;
import java.io.FileInputStream;
```

```
import java.io.IOException;

public class ExcelUtil {

    /**
     * Parses an Excel sheet and converts row data into a 2D Object array for TestNG DataProvider.
     */
    public static Object[][] getTestData(String filePath, String sheetName) {
        Object[][] data = null;

        try (FileInputStream fis = new FileInputStream(filePath);
             Workbook workbook = WorkbookFactory.create(fis)) {

            Sheet sheet = workbook.getSheet(sheetName);
            if (sheet == null) {
                throw new RuntimeException("Sheet '" + sheetName + "' not found in " + filePath);
            }

            int rowCount = sheet.getLastRowNum();
            int colCount = sheet.getRow(0).getLastCellNum();

            data = new Object[rowCount][colCount];

            DataFormatter formatter = new DataFormatter();

            for (int i = 0; i < rowCount; i++) {
```

```
            Row row = sheet.getRow(i + 1); // Skip header row
            for (int j = 0; j < colCount; j++) {
                if (row == null) {
                    data[i][j] = "";
                } else {
                    Cell cell = row.getCell(j);
                    data[i][j] = formatter.formatCellValue(cell);
                }
            }
        }

    } catch (IOException e) {
        Log.error("Failed to read Excel data from path: " + filePath);
        throw new RuntimeException("Excel Read Exception: ", e);
    }

    return data;
  }
}
```

4.6. Page Objects (BasePage & Sample Page)

src/main/java/com/framework/pages/BasePage.java

```
package com.framework.pages;

import com.framework.config.ConfigReader;
import com.framework.driver.DriverManager;
```

```
import com.framework.utils.Log;
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;

public abstract class BasePage {
    protected WebDriverWait wait;

    public BasePage() {
        int explicitWait = ConfigReader.getIntProperty("explicitWait");
        this.wait = new WebDriverWait(DriverManager.getDriver(),
Duration.ofSeconds(explicitWait));
    }

    protected void click(By locator, String elementName) {
        WebElement element = wait.until(ExpectedConditions.elementToBeClickable(locator));
        element.click();
        Log.info("Clicked on element: " + elementName);
    }

    protected void sendKeys(By locator, String value, String elementName) {
        WebElement element = wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
        element.clear();
```

```
        element.sendKeys(value);
        Log.info("Typed '" + value + "' into: " + elementName);
    }

    protected String getText(By locator, String elementName) {
        WebElement element = wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
        String text = element.getText();
        Log.info("Retrieved text from " + elementName + ": " + text);
        return text;
    }

    protected boolean isDisplayed(By locator, String elementName) {
        try {
            boolean displayed =
wait.until(ExpectedConditions.visibilityOfElementLocated(locator)).isDisplayed();
            Log.info("Visibility check for " + elementName + ": " + displayed);
            return displayed;
        } catch (Exception e) {
            Log.warn("Element not visible: " + elementName);
            return false;
        }
    }
}
```

src/main/java/com/framework/pages/LoginPage.java

```
package com.framework.pages;
```

```
import org.openqa.selenium.By;

public class LoginPage extends BasePage {

    // Locators using By strategy for better thread safety and dynamic waits
    private final By txtUsername = By.name("username");
    private final By txtPassword = By.name("password");
    private final By btnLogin = By.xpath("//button[@type='submit']");
    private final By txtDashboardHeader = By.xpath("//h6[text()='Dashboard']");
    private final By txtErrorMessage = By.xpath("//p[contains(@class, 'alert-content-text')]");

    public LoginPage enterUsername(String username) {
        sendKeys(txtUsername, username, "Username Field");
        return this;
    }

    public LoginPage enterPassword(String password) {
        sendKeys(txtPassword, password, "Password Field");
        return this;
    }

    public LoginPage clickLogin() {
        click(btnLogin, "Login Button");
        return this;
    }
```

```
    public boolean isDashboardDisplayed() {
        return isDisplayed(txtDashboardHeader, "Dashboard Header");
    }

    public String getErrorMessage() {
        return getText(txtErrorMessage, "Error Message");
    }
}
```

4.7. Test Framework & Listeners

src/test/java/com/framework/listeners/TestListener.java

```
package com.framework.listeners;

import com.aventstack.extentreports.ExtentTest;
import com.aventstack.extentreports.MediaEntityBuilder;
import com.aventstack.extentreports.Status;
import com.framework.reports.ExtentManager;
import com.framework.reports.ExtentTestManager;
import com.framework.utils.Log;
import com.framework.utils.ScreenshotUtil;
import org.testng.ITestContext;
import org.testng.ITestListener;
import org.testng.ITestResult;

public class TestListener implements ITestListener {
```

```
    @Override
    public void onStart(ITestContext context) {
        Log.info("=== Starting Test Suite Execution: " + context.getName() + " ===");
    }

    @Override
    public void onFinish(ITestContext context) {
        Log.info("=== Finished Test Suite Execution: " + context.getName() + " ===");
        ExtentManager.getInstance().flush();
    }

    @Override
    public void onTestStart(ITestResult result) {
        Log.info("Starting Test: " + result.getMethod().getMethodName());
        ExtentTest test =
ExtentManager.getInstance().createTest(result.getMethod().getMethodName());
        ExtentTestManager.setTest(test);
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        Log.info("Test PASSED: " + result.getMethod().getMethodName());
        ExtentTestManager.getTest().log(Status.PASS, "Test Executed Successfully");
        ExtentTestManager.unload();
    }
```

```
    @Override
    public void onTestFailure(ITestResult result) {
        Log.error("Test FAILED: " + result.getMethod().getMethodName());
        Log.error(result.getThrowable().getMessage());

        String base64Screenshot = ScreenshotUtil.getBase64Screenshot();

        ExtentTestManager.getTest().log(
            Status.FAIL,
            result.getThrowable(),
            MediaEntityBuilder.createScreenCaptureFromBase64String(base64Screenshot).build()
        );
        ExtentTestManager.unload();
    }

    @Override
    public void onTestSkipped(ITestResult result) {
        Log.warn("Test SKIPPED: " + result.getMethod().getMethodName());
        ExtentTestManager.getTest().log(Status.SKIP, "Test Skipped: " + result.getThrowable());
        ExtentTestManager.unload();
    }
}
```

src/test/java/com/framework/tests/BaseTest.java

```
package com.framework.tests;
```

```
import com.framework.config.ConfigReader;
import com.framework.driver.DriverFactory;
import com.framework.driver.DriverManager;
import com.framework.utils.Log;
import org.openqa.selenium.WebDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

public abstract class BaseTest {

    @BeforeMethod
    public void setUp() {
        WebDriver driver = DriverFactory.createDriver();
        DriverManager.setDriver(driver);
        Log.info("Driver initialized on Thread ID: " + Thread.currentThread().getId());

        String baseUrl = ConfigReader.getProperty("baseUrl");
        DriverManager.getDriver().get(baseUrl);
        Log.info("Navigated to: " + baseUrl);
    }

    @AfterMethod
    public void tearDown() {
        if (DriverManager.getDriver() != null) {
            DriverManager.getDriver().quit();
```

```
            Log.info("Driver closed on Thread ID: " + Thread.currentThread().getId());
            DriverManager.unload();
        }
    }
}
```

src/test/java/com/framework/tests/LoginTest.java

```
package com.framework.tests;

import com.framework.pages.LoginPage;
import com.framework.utils.ExcelUtil;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class LoginTest extends BaseTest {

    @DataProvider(name = "excelData", parallel = false)
    public Object[][] getLoginData() {
        String excelPath = "src/test/resources/testdata/TestData.xlsx";
        return ExcelUtil.getTestData(excelPath, "LoginData");
    }

    @Test(dataProvider = "excelData")
    public void testDataDrivenLogin(String username, String password, String expectedResult) {
        LoginPage loginPage = new LoginPage();
```

```
        loginPage.enterUsername(username)
                .enterPassword(password)
                .clickLogin();

        if (expectedResult.equalsIgnoreCase("Success")) {
            Assert.assertTrue(loginPage.isDashboardDisplayed(), "Dashboard was expected but not displayed.");
        } else {
            Assert.assertTrue(loginPage.getErrorMessage().contains("Invalid"), "Expected error message not displayed.");
        }
    }
}
```

4.8. Suite Execution Runner

src/test/resources/testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Data Driven Automation Suite" parallel="methods" thread-count="3">
    <listeners>
        <listener class-name="com.framework.listeners.TestListener"/>
    </listeners>

    <test name="Login Tests">
        <classes>
            <class name="com.framework.tests.LoginTest"/>
```

```
    </classes>
  </test>
</suite>
```

5. Design Choice Analysis & Alternatives

1. ThreadLocal<WebDriver> vs. Static Driver Instances

Why this choice: Using a single public static WebDriver driver causes race conditions in multi-threaded executions. ThreadLocal wraps every driver instance into a dedicated memory bucket mapped to the executing thread, enabling safe parallel execution.

Alternative: Dependency Injection (e.g., Guice or Spring Test) passing WebDriver around via constructors. While clean, it adds dependency weight; ThreadLocal achieves isolation natively with minimal overhead.

2. Standard By Locators vs. @FindBy PageFactory

Why this choice: PageFactory initialized via PageFactory.initElements() caches elements or relies on implicit waits, which can lead to StaleElementReferenceException when combined with modern dynamic SPA (Single Page Application) frameworks. Using explicit waits on raw By locators ensures fresh DOM queries every time an interaction occurs.

Alternative: Fluent Page Object Model with @FindBy. Operates well for simple apps, but struggles under fast-changing DOM states compared to explicit WebDriverWait.

3. Base64 Screenshots vs. Local PNG File Storage

Why this choice: Saving screenshots as Base64 strings enables ExtentReports to embed images directly into a single self-contained .html report file. This simplifies report sharing across CI/CD pipelines and team channels without dealing with broken image links.

Alternative: Saving .png files to disk and embedding relative file paths. While this saves memory on massive runs, it breaks if the HTML file is emailed or moved without its image folder.

6. How to Run the Framework

1. Place your Excel sheet at src/test/resources/testdata/TestData.xlsx with a sheet named LoginData structured like:

Username Password ExpectedResult

Admin admin123 Success

InvalidUser wrongPass Fail

2. Execute default (QA) suite via Maven:

Bash

mvn clean test

3. Execute against Dev environment with system property override:

Bash

mvn clean test -Denv=dev

4. View Execution Reports:

Open build/reports/ExtentReport.html in your browser.

Activity 2 - Chain of Thought (CoT)

Here is the systematic Chain of Thought (CoT) reasoning process used to investigate and uncover the root causes behind the generic error: "Transaction could not be completed.

"Chain of Thought (CoT) Investigation

Step 1: Define the Transaction Lifecycle

To diagnose why a ₹50,000 transfer failed, we must trace the request path:

Customer/App > Auth/Security > Network/Switch > Core Banking System > Beneficiary Account

Every layer in this workflow introduces potential points of failure.

Step 2: Examine Account & Balance Constraints (Remitter Side)

Thought: Before sending the request out to the bank servers, the system evaluates the sender's account state and transfer capability for ₹50,000.

Question: Does the user have the money, and are they allowed to send this amount today?

Scenario A: The account balance is less than ₹50,000 (e.g., balance is ₹35,000).

💡 Discovery Uncovered: Insufficient balance

Scenario B: The user has enough money, but they have already transferred their maximum permitted daily threshold (e.g., daily limit is ₹1,000,000, but ₹960,000 was spent earlier).

💡 Discovery Uncovered: Daily limit exceeded

Step 3: Examine Authentication & Authorization Layer

Thought: High-value transactions (₹50,000) require Two-Factor Authentication (2FA) via OTP or T-PIN.

Question: Was authorization completed successfully within the required timeframe?

Observation: If the user took too long to enter the One-Time Password sent to their mobile, or entered a delayed payload, the validation token becomes invalid.

💡 Discovery Uncovered: OTP expired

Step 4: Examine Fraud & Security Engines

Thought: Banking platforms pass high-value transfers through automated risk engines before processing.

Question: Did the ₹50,000 transfer trigger security anomalies?

Observation: Factors like a new device login, unusual location/IP, rapid sequential transfers, or a newly added beneficiary within the 24-hour cooling window can flag the transaction as high risk.

💡 Discovery Uncovered: Fraud rule triggered

Step 5: Examine Network & Transport Mechanics

Thought: The app issues an API request across the public cellular/Wi-Fi network to the bank's gateway.

Question: Did the request payload reach the destination and receive a response in time?

Observation: Poor network signal or high latency causes the connection to drop mid-flight or time out before the client gets a HTTP 200/ACK response.

💡 Discovery Uncovered: Network timeout

Step 6: Examine Payment Middleware & Switch Infrastructure

Thought: In India (indicated by ₹ symbol), ₹50,000 transfers typically route through payment switches like UPI, IMPS, or NEFT.

Question: Is the central routing switch operational?

Observation: If the NPCI UPI switch or the underlying bank payment gateway switch experiences heavy traffic or service failure, routing fails instantly.

💡 Discovery Uncovered: UPI switch unavailable

Step 7: Examine Bank Server & Core Banking Infrastructure (CBS)

Thought: Assuming the switch routed the request, the bank’s internal servers must process the double-entry accounting ledger.

Question: Are the bank's API servers and backend databases online?

Scenario A: The bank’s external API application servers are offline or unresponsive.

💡 Discovery Uncovered: Bank server unavailable

Scenario B: The centralized Core Banking System (CBS) database handling ledger updates is undergoing scheduled maintenance or experiencing a database crash.

💡 Discovery Uncovered: Core banking unavailable

Step 8: Examine Beneficiary (Receiving Account) State

Thought: The remitter's side and payment pipe cleared successfully, but the money needs to land in the target account.

Question: Can the destination account accept ₹50,000?

Observation: If the target account is dormant, frozen by regulatory orders, blocked due to uncompleted KYC, or closed, the transaction gets rejected by the receiving node.

💡 Discovery Uncovered: Beneficiary inactive

Summary of Discoveries

Phase	Technical Layer	Root Cause Discovered
1	Sender Account	1. Daily limit exceeded
		2. Insufficient balance
2	Auth & Security	3. OTP expired
		4. Fraud rule triggered
3	Transport & Network	5. Network timeout
4	Middleware / Routing	6. UPI switch unavailable
5	Bank Backend	7. Bank server unavailable
		8. Core banking unavailable

| 6 | Receiver Side | 9. Beneficiary inactive |

1. Scenario Name : Scenario 1 — Search Functionality

Weak Prompt : "Test the search feature."

What I Feel is Missing:

1. Search Engine: Which backend service or product should power the search?

2. UI Component: Should the search be rendered as a persistent text box or a search icon?

3. Placement: Where should the search element be positioned in the UI?

4. Input Constraints: What are the accepted inputs (alphanumeric, special characters) and case-sensitivity rules?

5. Search Logic: Should the query execute an exact match or a partial ("contains") match?

✅ Rewritten Prompt

Persona: Act as Senior QA Engineer

Context: Use the Amazon website from India.

Instruction: Generate test case for UI search feature

[Critical] Where the Search box is positioned, Use alphanumeric, special characters during the search, whether the search result returns exact or partial match results.

[Mandatory] Search the books and educational toys for 3 years old kid with less than 1000 rupees.

[Strict] Generate 10 test case to cover all of the above features.

Example: Test ID | Description | Test Data | Pre-Condition | Steps | Expected Result

TC01 | Verify Search feature | Laptop | Amazon.in should login | 1. Login to Amazon.in | Login should successful

Output Format: Generate test cases to paste the table in Notepad++

Tone: It should meat the QA standard and should be simple & professional

Weak prompt Output:

Search Functionality

"Test the search feature."

Test Status: Passed ✅

The live web search integration is functional and returning current results.

Test Query Executed

Query: latest tech news August 2026

Status: 200 OK (Data retrieved successfully)

Sample Search Results

OpenAI / AI Updates: Reporting on OpenAI's Astra model, GPT-5.5 releases, and security safety pauses.

Emerging Tech Trends: World Economic Forum highlights including everything-to-grid energy and direct lithium extraction.

The World Economic Forum

Open-Source & Wikimedia: Tech updates regarding client-side rendering and category sorting features.

Diff - Wikimedia

ICEPOT Output:

Here is the test suite containing 10 comprehensive test cases designed to QA standards for the Amazon India (Amazon.in) search feature.

Test ID	Description	Test Data	Pre-Condition	Steps	Expected Result
TC01	Verify UI positioning and visibility of the Search box across main pages	N/A	Amazon.in homepage is loaded	1. Open Amazon.in homepage. 2. Verify the location of the search input box. 3. Navigate to a category page and a product detail page.	Search box is prominently positioned at the top header center across all pages, with category dropdown on the left and search button on the right.
TC02	Verify search functionality using alphanumeric characters	"Lego classic 10696"	Amazon.in homepage is loaded	1. Enter "Lego classic 10696" in the search box.	Search

results page loads successfully displaying Lego sets matching model number 10696.
|

	2. Click the Search icon or press Enter.

| TC03 | Verify search handling when input contains special characters | "C# programming book & guide @2026!" | Amazon.in homepage is loaded | 1. Enter special character string in search box.
2. Click Search button. | System handles special characters gracefully without application errors or blank screens, returning relevant C# books. |

| TC04 | Verify Exact Match search result accuracy | "The Very Hungry Caterpillar board book" | Amazon.in homepage is loaded | 1. Enter exact book title in the search box.
2. Click Search icon.
3. Inspect the top search result. | The exact matching title "The Very Hungry Caterpillar" appears at the top position of the search results page. |

| TC05 | Verify Partial Match search result accuracy | "wooden toy train" | Amazon.in homepage is loaded | 1. Enter partial product keywords in search box. | Search results show items matching any or all keywords ("wooden", "toy", "train") with high relevance ranking. |
| | | | | 2. Click Search icon. | |

TC06	Verify search for Books for 3-year-old kids priced under ₹1000	Search Query: "story books for 3 year old kids", Filter: Under ₹1,000	Amazon.in homepage is loaded	1. Select "Books" in category dropdown.	Displayed books are strictly age-appropriate for 3-year-olds with prices equal to or less than ₹1,000.
				2. Enter "story books for 3 year old kids" and search.	
				3. Apply Price Filter "Under ₹1,000" from left panel.	

| TC07 | Verify search for Educational Toys for 3-year-old kids priced under ₹1000 | Search Query: "educational toys for 3 year old kids", Price Filter: Max ₹1000 | Amazon.in homepage is loaded | 1. Type "educational toys for 3 year old kids under 1000" in search bar. | Search results display educational toys (puzzles, building blocks, learning sets) suited for 3-year-olds with prices under |

| | | |
| | | 2. Press Enter. | |
₹1,000. |
| | | |
| | | 3. Check price filter in left navigation rail.
| |

+---------+---+-------------
---+-----------------------------+----------
---+-------------------
---+

| TC08 | Verify category pre-selection combined with age and price constraints
| Category: Toys & Games, Search Term: "learning toys 3 years", Filter: Up to ₹1000 |
Amazon.in homepage is loaded | 1. Select "Toys & Games" from category dropdown.
| System filters out non-toy categories and shows 3-year learning toys costing ≤ ₹1,000.
|

| | | |
| | | 2. Search for "learning toys 3 years".
| |

| | | |
| | | 3. Select "Up to ₹1,000" under Price filter.
| |

+---------+---+-------------
---+-----------------------------+----------
---+-------------------
---+

| TC09 | Verify search query combining Alphanumeric text, Special Characters, and Price/Age
terms | "3+ yr old kid's learning toy & puzzle <1000" |
Amazon.in homepage is loaded | 1. Enter complex combination query containing symbols (+, ',
&, <) and numbers in search box. | Search query executes without script errors and
returns relevant learning/puzzle toys targeted at 3+ year olds under ₹1,000. |

| | | |
| | | 2. Click Search. | |
|

TC10	Verify Search box persistence, clearing, and re-querying capability	"puzzle toys for 3 year old"	Search executed on Amazon.in	1. Perform search.	Query string persists in search box on results page, clear button resets input field instantly, and new suggestions appear
				2. Check if search term remains in search box.	when typing.
				3. Click 'X' (clear) button in search box.	
				4. Enter new query.	

Key Coverage Summary

* UI Positioning (TC01): Header search bar layout verification across page views.

* Input Types (TC02, TC03, TC09): Alphanumeric, special characters (#, @, &, +, <), and mixed syntax.

* Matching Precision (TC04, TC05): Exact product title matches vs keyword partial matches.

* Business Mandatory Scenarios (TC06, TC07, TC08): Age-specific (3 years old) books and educational toys priced under ₹1,000 using search queries and facet filters.

Here is a direct side-by-side comparison of how your Input 1 (Vague) vs. Input 2 (Detailed Prompt Engineering) transformed the AI's understanding and generated output.

Side-by-Side Prompt & Output Comparison

Comparison Axis	Input 1: Bare Request	Input 2: Structured Prompting
Prompt Text	Search Functionality "Test the search feature."	Persona: Senior QA EngineerContext: Amazon.inInstructions: [Critical], [Mandatory], [Strict] rulesExample Schema & Format: Defined
Prompt Technique	Zero-Context / Undefined Intent: No role, no target system, no format, no constraints.	Role-Task-Context (RTC) Framework: Uses explicit Persona, domain constraints, priority tags, and output formatting.
AI Interpretation	Interpreted literally as: "Test whether AI search functionality works right now."	Interpreted as: "Write a professional 10-case QA test suite for Amazon India's UI search bar."
Generated Output	Executed a web search query and returned a system test status report.	Generated a 10-row structured QA test suite with realistic test data, exact matches, price filters (₹1,000), and special characters.
Usability	Unusable for a software testing workflow.	Ready to paste directly into Notepad++, Excel, or TestRail.

2. Scenario Name : Scenario 2 — File Upload

Weak Prompt : "Generate test cases for file upload."

What I Feel is Missing:

1. Where to upload the file?
2. What are the file formats will it accept like PDF, TIF, TXT, DOCX, XSLX, HTM?
3. What is the maximum file size?
4. Will it accept the compressed files like zip, 7z?
5. Will the file names accept the long names with special charaters?

✅ Rewritten Prompt

Persona: Act as Senior QA Engineer

Context: Use the ilovepdf site to upload the files.

Instruction: Generate test case for file upload feature

[Critical] Where the upload file icon is positioned, Upload the files and if it is uploaded successfully then the success message should display and error should display if file size exceed or incorrect formats or very long file names.

[Mandatory] Include the following file formats PDF, TIF, TXT, DOCX, XSLX, HTM.

[Important] File size should not exceed greater than 5 MB.

It should not accept the compressed files.

It should not allow the very long file names like more than 55 characters.

It should allow the special characters in the file names.

[Strict] Generate 15 test case to cover positive, negative, edge and boundary.

Example: Test ID | Description | Test Data | Pre-Condition | Steps | Expected Result

TC01 | Verify Search feature | Laptop | Amazon.in should login | 1. Login to Amazon.in | Login should successful

Output Format: Generate test cases in ASCII grid table format to paste the table in Notepad++.

Tone: It should be in simple and formal.

Weak prompt Output:

Here's a comprehensive set of test cases for File Upload functionality, covering positive, negative, validation, security, UI, and performance scenarios.

TC ID Test Case Steps / Input Expected Result Priority

TC-01 Upload valid file Upload a supported file with valid size File uploads successfully High

TC-02 Upload multiple valid files Select multiple supported files All files upload successfully if multiple uploads are supported High

TC-03 Upload unsupported file type Upload .exe / unsupported extension Upload is rejected with appropriate error High

TC-04 Upload maximum allowed file size Upload file exactly at size limit File uploads successfully High

TC-05 Exceed maximum file size Upload file larger than allowed limit Upload is rejected with clear error message High

TC-06 Upload empty/0-byte file Upload a 0-byte file System handles it according to requirements; appropriate validation shown Medium

TC-07 Upload file with special characters Use filename such as test@#$%.pdf File is handled safely and uploaded/renamed as expected Medium

TC-08 Upload file with spaces Upload My Test File.pdf File uploads successfully and filename is preserved correctly Medium

TC-09 Upload duplicate file Upload the same file twice System handles duplicates according to requirements Medium

TC-10 Cancel file selection Select a file, then cancel the file picker No file is uploaded or selected Medium

TC-11 Remove selected file Select a file and click Remove/Delete File is removed from the upload list High

TC-12 Replace uploaded file Upload file, then select another file Previous file is replaced if replacement is supported Medium

TC-13 Upload without selecting file Click Upload/Submit without a file Appropriate validation message is displayed if file is mandatory High

TC-14 Upload corrupted file Upload a corrupted PDF/image/document System rejects it or handles it gracefully Medium

TC-15 Upload wrong MIME type Rename an unsupported file with a supported extension Server validates actual file type and rejects it if invalid High

TC-16 Upload file with long filename Use filename exceeding supported length Filename is validated/truncated appropriately; upload does not break UI Medium

TC-17 Upload Unicode filename Upload résumé_日本語.pdf File uploads and filename is handled correctly Medium

TC-18 Upload file with uppercase extension Upload TEST.PDF File is accepted if extension is supported Low

TC-19 Upload valid image Upload supported JPG/PNG image Image uploads successfully and preview appears if supported High

TC-20 Upload invalid image Upload unsupported image format Appropriate validation error is displayed Medium

TC-21 Upload via drag and drop Drag a valid file into upload area File is selected/uploaded successfully High

TC-22 Drag unsupported file Drag an unsupported file into upload area File is rejected with appropriate message Medium

TC-23 Verify upload progress Upload a relatively large valid file Progress indicator is displayed accurately Medium

TC-24 Network interruption Disconnect network during upload Upload fails gracefully and user receives an appropriate message High

TC-25 Retry failed upload Cause upload failure and click Retry Upload resumes/restarts successfully if supported Medium

TC-26 Refresh during upload Refresh page while upload is in progress System handles incomplete upload without corruption Medium

TC-27 Session timeout during upload Allow session to expire during upload Upload is rejected safely and user is prompted appropriately High

TC-28 Verify uploaded file Upload file and download/open it afterward Retrieved file matches the original file High

TC-29 Verify file metadata Upload file and inspect displayed name/size/type Metadata is accurate Medium

TC-30 Security: executable upload Attempt to upload executable content File is blocked according to security policy Critical

TC-31 Security: malicious filename Upload filename containing path traversal characters such as ../../file.pdf Path is sanitized; arbitrary filesystem access is impossible Critical

TC-32 Security: malicious content Upload a file containing potentially malicious content File is scanned/rejected according to security requirements Critical

TC-33 Authentication check Attempt upload without authentication Upload is denied if authentication is required Critical

TC-34 Authorization check User without upload permission attempts upload Upload is denied Critical

TC-35 Concurrent uploads Upload several files simultaneously System processes uploads correctly without data corruption High

TC-36 Maximum number of files Upload more files than the allowed count Additional files are rejected with clear message High

TC-37 File type validation message Upload invalid extension Error identifies allowed file types Medium

TC-38 File size validation message Upload oversized file Error identifies maximum allowed size Medium

TC-39 Browser compatibility Test upload in supported browsers Upload works consistently across supported browsers Medium

TC-40 Mobile upload Upload using mobile browser/device File selection/camera/gallery integration works as expected Medium

TC-41 UI during upload Start upload and inspect controls Buttons/status indicators behave correctly and prevent unintended duplicate submissions Medium

TC-42 Upload button state Select invalid/valid files Upload button is enabled/disabled according to validation rules Medium

TC-43 Accessibility Navigate upload control using keyboard File upload can be operated using keyboard/accessibility controls Medium

TC-44 Screen reader support Use screen reader with upload control Labels, errors, progress, and status are announced appropriately Medium

TC-45 Server-side validation Bypass client-side validation and send invalid file directly Server still validates file type, size, permissions, etc.Critical

ICEPOT Output:

Below are 15 file-upload test cases for the iLovePDF site, written from a Senior QA Engineer perspective. The cases cover positive, negative, edge, and boundary conditions based on your specified requirements. iLovePDF supports file selection through the upload area and also supports drag-and-drop.

+--------+---------------------------------------+---------------------+---------------------+---+----------+

| Test ID| Description | Test Data | Pre-Condition | Steps | Expected Result | Type |

+--------+---------------------------------------+---------------------+---------------------+---+----------+

| TC01 | Verify upload file icon/area | Valid PDF file | iLovePDF page opened | 1. Open iLovePDF upload page | Upload file icon/area should be clearly visible | Positive |

| | is displayed | | | 2. Locate upload icon/area | and positioned correctly for file selection. | |

+--------+----------+----------+----------+----------+----------+

| TC02 | Upload valid PDF file | test.pdf, 4 MB | Upload area visible | 1. Click upload/select file | PDF file should upload successfully and success | Positive |

| | | | | 2. Select test.pdf | message should be displayed. | |

+--------+----------+----------+----------+----------+----------+

| TC03 | Upload valid TIF file | test.tif, 4 MB | Upload area visible | 1. Click upload/select file | TIF file should upload successfully and success | Positive |

| | | | | 2. Select test.tif | message should be displayed. | |

+--------+----------+----------+----------+----------+----------+

| TC04 | Upload valid TXT file | test.txt, 4 MB | Upload area visible | 1. Click upload/select file | TXT file should upload successfully and success | Positive |

| | | | | 2. Select test.txt | message should be displayed. | |

+--------+----------+----------+----------+----------+----------+

| TC05 | Upload valid DOCX file | test.docx, 4 MB | Upload area visible | 1. Click upload/select file | DOCX file should upload successfully and success | Positive |

| | | | | 2. Select test.docx | message should be displayed. | |

+--------+----------+----------+----------+----------+----------+

| TC06 | Upload valid XLSX file | test.xlsx, 4 MB | Upload area visible | 1. Click upload/select file | XLSX file should upload successfully and success | Positive |

| | | | | 2. Select test.xlsx | message should be displayed. | |

+--------+----------+----------+----------+----------+----------+----------+

| TC07 | Upload valid HTM file | test.htm, 4 MB | Upload area visible | 1. Click upload/select file | HTM file should upload successfully and success | Positive |

| | | | | 2. Select test.htm | message should be displayed. | |

+--------+----------+----------+----------+----------+----------+----------+

| TC08 | Verify maximum file size boundary | test.pdf, exactly | Upload area visible | 1. Select file of exactly 5 MB | File of exactly 5 MB should be accepted and | Boundary |

| | | 5 MB | | 2. Start upload | success message should be displayed. | |

+--------+----------+----------+----------+----------+----------+----------+

| TC09 | Verify file size greater than | test.pdf, 5.01 MB | Upload area visible | 1. Select file greater than 5 MB | File should not be uploaded and an appropriate | Negative |

| | allowed limit | | | 2. Start upload | file-size error message should be displayed. | |

+--------+----------+----------+----------+----------+----------+----------+

| TC10 | Verify compressed files are not | test.zip, 2 MB | Upload area visible | 1. Select ZIP file | Compressed file should be rejected and an appropriate| Negative |

| | accepted | | | 2. Start upload | unsupported/compressed-file error should display. | |

+--------+----------+----------+----------+----------+----------+----------+

| TC11 | Verify unsupported file format | test.exe, 2 MB | Upload area visible | 1. Select EXE file | Unsupported file format should be rejected and an | Negative |

| | | | | 2. Start upload | appropriate format error should be displayed. | |

+--------+----------+----------+----------+----------+----------+----------+

| TC12 | Verify filename length boundary | 55-character | Upload area visible | 1. Select supported file having | File with filename of exactly 55 characters should | Boundary |

| | | filename.pdf, 3 MB | | 55-character filename | be accepted if 55 is the maximum allowed length. | |

| | | | | 2. Start upload | Success message should be displayed. | |

+--------+----------+----------+----------+----------+----------+----------+

| TC13 | Verify filename greater than | 56-character | Upload area visible | 1. Select supported file having | File with filename greater than 55 characters should | Negative |

| | allowed length | filename.pdf, 3 MB | | 56-character filename | be rejected and filename-length error should display.| |

| | | | | 2. Start upload | | |

+--------+----------+----------+----------+----------+----------+----------+

| TC14 | Verify special characters in | test@#$%_file!.pdf | Upload area visible | 1. Select supported file with | File should be accepted and uploaded successfully. | Positive |

| | filename are allowed | 3 MB | | special characters in filename | Success message should be displayed and filename | |

| | | | | 2. Start upload | should be handled/displayed correctly. | |

+--------+----------+----------+----------+----------+----------+----------+

| TC15 | Verify combined boundary and | test@#$%_file.pdf, | Upload area visible | 1. Select supported file | File should be rejected because its size is greater | Edge |

| | validation conditions | 5.01 MB, 56+ chars | | 2. Start upload | than 5 MB and/or filename exceeds 55 characters. | |

| | | | | 3. Observe validation message | Appropriate error message(s) should be displayed. | |

+--------+----------+----------+----------+----------+----------+

Coverage

Positive: TC01–TC07, TC14

Boundary: TC08, TC12

Negative: TC09–TC11, TC13

Edge: TC15

Mandatory formats covered: PDF, TIF, TXT, DOCX, XLSX, HTM

Maximum size: 5 MB

Compressed files: ZIP rejection

Unsupported format: EXE rejection

Filename limit: 55 characters

Special characters: Allowed

Upload success/error messages: Covered

Upload icon/area positioning: TC01

Note: I treated 5 MB and 55 characters as inclusive limits—so exactly 5 MB and exactly 55 characters should pass, while values above those limits should fail. This follows your stated requirement;

the live iLovePDF documentation itself currently shows different size limits for various tools, so these test cases should be treated as the requirements for the application under test rather than as claims about iLovePDF's current global limits.

Observation	First Output	Second Output	What Improved & Why
1. Requirement coverage	Covered general file-upload scenarios, including security, browser compatibility, accessibility, and network failures. However, it did not specifically focus on all the requirements from the prompt.	Explicitly covers **PDF, TIF, TXT, DOCX, XLSX, HTM**, 5 MB limit, compressed files, filename length, special characters, and upload messages.	**Improved requirement traceability.** The second output maps the test cases directly to the stated business requirements, making it easier for QA to verify complete coverage.
2. Boundary and negative testing	Included many useful negative and edge cases such as 0-byte files, corrupted files, oversized files, MIME spoofing, and long filenames.	Specifically tests **exactly 5 MB vs. greater than 5 MB** and **exactly 55 vs. greater than 55 characters**, along with unsupported and compressed files.	**Improved boundary testing.** Explicit boundary values make the expected behavior unambiguous and help detect off-by-one validation defects.
3. Format and execution clarity	Provided 45 detailed scenarios, but the large scope could make it harder to execute against the specific requirement.	Reduced to exactly **15 prioritized test cases** and added a **Type** column for Positive, Negative, Boundary, and Edge cases.	**Improved usability and focus.** The second output follows the requested 15-case limit and gives testers a clear classification of each scenario, making execution and coverage review easier.

3. Scenario Name : Scenario 6 — Mobile App (OTP Login)

Weak Prompt : "Test OTP login on mobile."

What I Feel is Missing:

1. Where it is Android or iPhone?
2. How many digits in the OTP like 4 or 6?
3. What is the OTP expiry time like 30 sec or 1 min?
4. Will it have resend OTP option?
5. When the networld fails to connect will the OTP resend automatically with any error?
6. If user moved to different country then how the OTP works?

✅ Rewritten Prompt

Persona: Act as Senior QA Engineer

Context: Use the WhatsApp login in iPhone using the OTP.

Instruction: Generate test case for WhatsApp login using the OTP.

[Critical] When the app installed it will ask the OTP for the registered mobile number.

Then OTP only accepts 6 digits with Resend OTP option.

[Mandatory] Include the OTP expiry time as 1 min and if user clicks the Resend OTP then it should send again or it will not.

[Essential] If user entered correct OTP but time expired then error message should display.

It user moved to different country then user should get a correct message.

It network connect to fails then user need to click Resend OTP button.

[Strict] Generate 7 test case to cover positive, negative, edge and boundary.

Example: Test ID | Description | Test Data | Pre-Condition | Steps | Expected Result

TC01 | Verify Search feature | Laptop | Amazon.in should login | 1. Login to Amazon.in | Login should successful

Output Format: Generate gherkin test cases.

Tone: It should be in simple and formal.

Weak prompt Output:

Test ID Description Test Data Pre-Condition Steps Expected Result Type

TC01 Verify OTP login screen is displayed Registered mobile number App is installed and user is logged out 1. Open mobile app 2. Enter registered mobile number 3. Tap Login/Continue OTP verification screen should be displayed and OTP should be sent to the registered mobile number Positive

TC02 Verify login with valid OTP Valid 6-digit OTP OTP has been received 1. Enter valid OTP 2. Tap Verify User should be successfully logged in and redirected to the home screen Positive

TC03 Verify login with invalid OTP Incorrect 6-digit OTP OTP verification screen is displayed 1. Enter incorrect OTP 2. Tap Verify Login should fail and an appropriate error message should be displayed Negative

TC04 Verify OTP with fewer digits 12345 OTP verification screen is displayed 1. Enter OTP with fewer than required digits 2. Tap Verify OTP should not be accepted and validation message should be displayed Negative

TC05 Verify OTP with more digits 1234567 OTP verification screen is displayed 1. Enter more than the allowed number of digits Extra digits should not be accepted or the user should receive appropriate validation Boundary

TC06 Verify expired OTP Expired OTP OTP has expired 1. Enter expired OTP 2. Tap Verify Login should fail and an OTP expired message should be displayed Negative

TC07 Verify resend OTP functionality New OTP OTP verification screen is displayed 1. Tap Resend OTP 2. Enter newly received OTP 3. Tap Verify New OTP should be sent and should allow successful login Positive

TC08 Verify old OTP after resend Old OTP New OTP has been generated 1. Request new OTP 2. Enter previous OTP 3. Tap Verify Previous OTP should be invalid and login should fail Negative

TC09 Verify OTP resend timer Resend timer OTP verification screen is displayed 1. Observe resend option 2. Wait until timer expires Resend option should remain disabled during the configured period and become available afterward Boundary

TC10 Verify OTP with blank input Blank OTP OTP verification screen is displayed 1. Leave OTP field blank 2. Tap Verify Login should not proceed and required-field validation should be displayed Negative

TC11 Verify OTP containing alphabets/special characters 12AB@# OTP verification screen is displayed 1. Enter alphabets/special characters Non-numeric characters should not be accepted if OTP is numeric Negative

TC12 Verify OTP auto-read functionality Valid OTP received through SMS SMS permission/OTP auto-read is supported 1. Receive OTP SMS 2. Observe OTP field OTP should be detected/auto-filled according to the app's supported behavior Positive

TC13 Verify OTP login with no network Valid OTP + no internet OTP screen is displayed 1. Disable network 2. Enter OTP 3. Tap Verify Login should fail gracefully with an appropriate network error message Negative

TC14 Verify multiple incorrect OTP attempts Multiple invalid OTPs OTP verification screen is displayed 1. Enter incorrect OTP repeatedly App should enforce configured attempt limits/security controls and display an appropriate message Security

TC15 Verify successful login session Valid OTP Valid OTP is available 1. Enter valid OTP 2. Tap Verify 3. Close and reopen app User should remain logged in according to the configured session policy Positive

ICEPOT Output:

Feature: WhatsApp Login Using OTP on iPhone

Background:

Given WhatsApp is installed on an iPhone

And the user has a registered mobile number

And the user is on the WhatsApp OTP verification screen

Scenario: TC01 - Verify OTP is requested after WhatsApp installation

Given WhatsApp has been installed for the first time

When the user enters the registered mobile number

Then WhatsApp should send an OTP to the registered mobile number

And the OTP verification screen should be displayed

Scenario: TC02 - Verify login with a valid 6-digit OTP

Given a valid OTP has been sent to the registered mobile number

And the OTP is within the 1-minute validity period

When the user enters the correct 6-digit OTP

Then the user should be successfully logged in to WhatsApp

Scenario: TC03 - Verify OTP accepts only 6 digits

Given the OTP verification screen is displayed

When the user enters fewer or more than 6 digits

Then the OTP should not be accepted

And the user should be prompted to enter a valid 6-digit OTP

Scenario: TC04 - Verify OTP expiry after 1 minute

Given a valid 6-digit OTP has been received

When 1 minute has elapsed

And the user enters the correct OTP

Then the OTP should not be accepted

And an appropriate OTP expired error message should be displayed

And the user should be provided with an option to request a new OTP

Scenario: TC05 - Verify Resend OTP functionality

Given the OTP verification screen is displayed

And the Resend OTP option is available

When the user clicks "Resend OTP"

Then a new OTP should be sent to the registered mobile number

And the new OTP should be valid for 1 minute

And the user should be able to log in using the new OTP

Scenario: TC06 - Verify OTP when network connection fails

Given the OTP verification screen is displayed

And the network connection fails

When the user attempts to verify the OTP

Then the OTP verification should not be completed

And an appropriate network error message should be displayed

And the user should be instructed to click "Resend OTP" after the network connection is restored

Scenario: TC07 - Verify login when user moves to another country

Given the user has moved to a different country

And the user is using the registered mobile number

When the user requests an OTP

Then WhatsApp should handle the country/phone number correctly

And the user should receive an appropriate message if the registered number or country is not supported

And the user should not be incorrectly logged in

Observation	First OTP Output	Second OTP Output	What Improved & Why
1. Requirement-specific coverage	The first output provided 15 generic OTP-login test cases, including blank OTP, invalid OTP, auto-read, session handling, and multiple attempts.	The second output focuses specifically on the requested **WhatsApp iPhone OTP flow**, including installation, registered number, 1-minute expiry, resend, network failure, and country change.	**Improved specificity.** The second output is better aligned with the exact business requirements instead of covering generic OTP functionality.
2. Boundary and expiry validation	Covered OTP length and expiry generally, but did not use the exact **1-minute expiry requirement**.	Explicitly validates **6-digit OTP** and **1-minute validity**, including the case where the correct OTP is entered after expiry.	**Improved boundary coverage.** Exact values make the test behavior measurable and help identify timing-related defects.
3. Format and readability	Used a traditional test-case table with columns such as Test ID, Description, Test Data, Steps, and Expected Result.	Uses **Gherkin Given/When/Then** scenarios with seven focused cases.	**Improved execution clarity.** Gherkin makes preconditions, actions, and expected behavior easy for QA, developers, and automation teams to understand and potentially automate.